Git & Maven
Interview Question Bank

Deduplicated, ranked by importance and grouped by topic
Prepared for a Test Automation Engineer with 4+ years of experience

Questions in your original document	**68**
Exact duplicates removed (same question, different wording of punctuation or case)	**0**
Near-duplicates merged (same question asked in different words)	**27**
Unique questions in this PDF	**41**

Section 1: Critical (Must Prepare)	16 questions
Section 2: High Priority	18 questions
Section 3: Medium Priority	7 questions

How to Use This Document

Questions are ordered by how likely they are to be asked and how much weight interviewers give them for a tester with 4+ years of experience. Work top to bottom: finish Section 1 before you spend time on Section 3 or 4. Inside each section the questions are grouped by topic, and within each topic the most important question comes first.

A red tag such as **[asked x3 in your bank]** means the same question showed up several times in your original document (in different wording). Repetition is a good signal that it is a frequently asked question.

Section	What it means
Critical (Must Prepare) 16 questions	Asked in almost every automation interview at 4+ years. Answer with how you use Git and Maven in YOUR framework and team.
High Priority 18 questions	Frequent practical questions. Answer with the command or pom.xml snippet you would use.
Medium Priority 7 questions	Occasionally asked, usually as follow-ups to a Git or Maven answer.

What Interviewers Look For (4+ Years)

Topic	What the interviewer is testing
Git: Merge, Rebase, Conflicts, PRs	Collaboration skills: when you rebase vs merge, how you resolve conflicts, and how PRs work in your team.
Git: Fundamentals & Workflow	Branching model, version-controlling the framework, Git vs GitHub, local vs remote.
Git: Everyday Commands	Fluent use of clone, add, commit, push, stash, revert and history commands.
Maven in Automation Projects	How Maven runs TestNG through Surefire, profiles per environment, and runs in Jenkins.
Maven: pom.xml & Dependencies	Structure of pom.xml, dependency scope, repositories, parent POM.
Maven: Lifecycle & Plugins	Lifecycle phases, clean/install/deploy, and what plugins do.

Note: short fragments in your file (for example 'git stash usage' or 'Profiles in Maven') are phrased as full questions and merged with matching ones. The Git and Maven parts of your file are ranked together but kept in separate topics.

Table of Contents

Section 1: Critical (Must Prepare) (16 questions) 5
Git: Merge, Rebase, Conflicts & Pull Requests (4) 5
Git: Fundamentals, Branching & Workflow (4) 5
Git: Everyday Commands, Undo & History (1) 6
Maven in Automation Projects (TestNG, Profiles, Jenkins) (2) 6
Maven: Fundamentals, pom.xml & Dependencies (3) 7
Maven: Lifecycle, Commands & Plugins (2) 7

Section 2: High Priority (18 questions) 9
Git: Merge, Rebase, Conflicts & Pull Requests (2) 9
Git: Fundamentals, Branching & Workflow (4) 9
Git: Everyday Commands, Undo & History (4) 10
Maven in Automation Projects (TestNG, Profiles, Jenkins) (2) 10
Maven: Fundamentals, pom.xml & Dependencies (2) 11
Maven: Lifecycle, Commands & Plugins (4) 11

Section 3: Medium Priority (7 questions) 13
Git: Fundamentals, Branching & Workflow (3) 13
Git: Everyday Commands, Undo & History (3) 13
Maven in Automation Projects (TestNG, Profiles, Jenkins) (1) 14

Topic Finder (all sections for one topic)

Use this if you want to study one topic at a time. Each number links to that topic's questions inside a section.

Topic	S1	S2	S3	Total
Git: Merge, Rebase, Conflicts & Pull Requests	4 Qs, p.5	2 Qs, p.9	-	**6**
Git: Fundamentals, Branching & Workflow	4 Qs, p.5	4 Qs, p.9	3 Qs, p.13	**11**
Git: Everyday Commands, Undo & History	1 Qs, p.6	4 Qs, p.10	3 Qs, p.13	**8**
Maven in Automation Projects (TestNG, Profiles, Jenkins)	2 Qs, p.6	2 Qs, p.10	1 Qs, p.14	**5**
Maven: Fundamentals, pom.xml & Dependencies	3 Qs, p.7	2 Qs, p.11	-	**5**
Maven: Lifecycle, Commands & Plugins	2 Qs, p.7	4 Qs, p.11	-	**6**

S1 = Critical, S2 = High, S3 = Medium, S4 = Low priority.

Section 1: Critical (Must Prepare) (16 questions)

Asked in almost every automation interview at 4+ years. Answer with how you use Git and Maven in YOUR framework and team.

Git: Merge, Rebase, Conflicts & Pull Requests (4)

1. What is the difference between git merge and git rebase? When do you use each? **[asked x2 in your bank]**

   > **Answer:** Merge joins two branches and keeps the full history, creating a merge commit. Rebase replays your commits on top of the target branch for a linear history but rewrites commits. Use merge for shared branches and rebase for cleaning up your own local feature branch (never rebase pushed, shared commits).
   >
   > **Example:** Bring the latest main into feature/checkout-tests before raising the PR.

   ```
   git checkout feature/checkout-tests
   git merge main        # keeps history, adds a merge commit
   git rebase main       # linear history, rewrites your commits
   ```

2. How do you resolve merge conflicts in Git? **[asked x2 in your bank]**

   > **Answer:** A conflict happens when two changes touch the same lines. Open the file, decide the correct code between the <<<<<<<, ======= and >>>>>>> markers, remove the markers, git add the file and finish with git commit (or git rebase --continue); then rerun the tests.
   >
   > **Example:** Two testers edited LoginPage.java locators; I merged both and ran the login tests before committing.

   ```
   git merge main
   # CONFLICT in LoginPage.java: edit the file, keep the right lines
   git add LoginPage.java
   git commit            # for a rebase: git rebase --continue
   ```

3. What is a pull request (PR) and why does QA use it? How do you create one? **[asked x2 in your bank]**

   > **Answer:** A pull request asks to merge a branch into another (usually main) after review. QA uses PRs to review test scripts, run CI checks and keep main stable. Create it by pushing your branch and opening Compare & pull request on GitHub/GitLab, adding a description and reviewers.
   >
   > **Example:** Every new page object goes through a PR with at least one reviewer and a passing smoke run.

   ```
   git push -u origin feature/payment-tests
   # open the PR in GitHub (or: gh pr create --title "Add payment tests" --base main)
   ```

4. What is the difference between git fetch and git pull? **[asked x2 in your bank]**

   > **Answer:** git fetch downloads remote changes without changing your working files. git pull is fetch plus merge (or rebase) into your current branch.
   >
   > **Example:** Fetch to see what teammates pushed to the test repo; pull to update before starting work.

   ```
   git fetch origin
   git pull origin main      # same as fetch + merge
   ```

Git: Fundamentals, Branching & Workflow (4)

5. What is Git and why is it important for QA engineers? What is the purpose of a version control tool? **[asked x3 in your bank]**

   > **Answer:** Git is a distributed version control system that tracks changes, supports branching and collaboration and lets you go back to earlier versions. QA needs it to store and review test scripts, work in parallel and run the code from CI.
   >
   > **Example:** The whole Selenium framework is in Git, so any build can be reproduced from a commit.

6. Explain the Git branching model for QA.

> **Answer:** A common model: main (stable), develop or release branches, and short-lived feature/bugfix branches per automation task merged back by pull requests; tags mark releases.
>
> **Example:** main > develop > feature/cart-tests; hotfix/locator-fix for urgent fixes.

```
git checkout develop
git checkout -b feature/cart-tests
# work, commit, push, then open a PR into develop
```

7. How do you version control your automation framework?

> **Answer:** Keep the framework in a Git repository with pom.xml, source, testng.xml and config, ignore build output and secrets with .gitignore, work through branches and pull requests, tag releases and let CI check out the repo.
>
> **Example:** The framework repo is cloned by Jenkins for every nightly run.

```
git init
git add .
git commit -m "Initial framework"
git remote add origin https://github.com/team/shop-automation.git
git push -u origin main
```

8. Which Git commands have you used? **[asked x2 in your bank]**

> **Answer:** Commonly used: clone, status, add, commit, push, pull, fetch, branch, checkout/switch, merge, rebase, stash, log, diff, reset, revert and tag. Mention what you really use daily.
>
> **Example:** Daily: pull, checkout -b, add, commit, push; weekly: rebase, stash, revert.

Git: Everyday Commands, Undo & History (1)

9. How do you revert a commit without losing history? **[asked x2 in your bank]**

> **Answer:** git revert creates a new commit that undoes the changes of an earlier commit, so history is kept (safe for pushed commits). Use -m 1 to revert a merge commit.
>
> **Example:** Revert a commit that broke the regression suite on main.

```
git revert <commit-id>
git revert -m 1 <merge-commit-id>   # for a merge commit
```

Maven in Automation Projects (TestNG, Profiles, Jenkins) (2)

10. How is Maven used in a Selenium or test automation framework?

> **Answer:** Maven manages dependencies (Selenium, TestNG, Rest Assured), builds the project, runs tests through the Surefire plugin, selects environments with profiles and runs on Jenkins from the command line.
>
> **Example:** mvn clean test -Pqa runs the shop regression on the QA environment.

```
<dependencies>
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId><artifactId>selenium-java</artifactId>
    <version>4.25.0</version>
  </dependency>
  <dependency>
    <groupId>org.testng</groupId><artifactId>testng</artifactId>
    <version>7.10.2</version><scope>test</scope>
  </dependency>
</dependencies>
```

11. How do you run TestNG tests from Maven (testng.xml with the Surefire plugin)?

> **Answer:** Configure the Maven Surefire plugin with the suite file and run mvn test. A property for the suite name lets Jenkins choose the suite from the command line.
>
> **Example:** mvn test -DsuiteXmlFile=smoke.xml runs only the smoke suite.

```
<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.2.5</version>
  <configuration>
    <suiteXmlFiles><suiteXmlFile>${suiteXmlFile}</suiteXmlFile></suiteXmlFiles>
  </configuration>
</plugin>
<!-- mvn clean test -DsuiteXmlFile=testng.xml -->
```

Maven: Fundamentals, pom.xml & Dependencies (3)

12. What is Maven and why is it used in automation frameworks? **[asked x2 in your bank]**

> **Answer:** Maven is a build and dependency management tool. In automation it downloads libraries automatically, gives a standard project structure, runs tests, creates reports and works easily with CI tools.
>
> **Example:** A new team member runs mvn clean test and gets all libraries without manual jar setup.

13. What is pom.xml and what is its structure? **[asked x2 in your bank]**

> **Answer:** pom.xml (Project Object Model) is the XML file that describes the project: coordinates (groupId, artifactId, version), packaging, properties, dependencies, build plugins, profiles, parent and repositories.
>
> **Example:** The pom lists Selenium and TestNG dependencies and the Surefire plugin.

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.shop</groupId><artifactId>shop-automation</artifactId><version>1.0</version>
  <properties><maven.compiler.source>17</maven.compiler.source></properties>
  <dependencies> ... </dependencies>
  <build><plugins> ... </plugins></build>
</project>
```

14. How does Maven manage dependencies? What are dependency scopes and how do you add a dependency from Maven Central? **[asked x3 in your bank]**

> **Answer:** Dependencies are declared in the pom; Maven downloads them (and their transitive dependencies) from Maven Central into the local ~/.m2 repository. Scopes: compile (default), provided, runtime, test, system, import. To add one, copy its XML from search.maven.org or mvnrepository.com into <dependencies>.
>
> **Example:** TestNG with test scope is available only for test code.

```
<dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>rest-assured</artifactId>
  <version>5.4.0</version>
  <scope>test</scope>
</dependency>
```

Maven: Lifecycle, Commands & Plugins (2)

15. What is the Maven lifecycle and what are its phases? **[asked x3 in your bank]**

> **Answer:** Maven has three lifecycles: default, clean and site. The default lifecycle phases are validate, compile, test, package, verify, install and deploy; running a phase runs all earlier phases.
>
> **Example:** mvn test runs validate, compile and test.

16. What are the common Maven commands (clean, install, test)?

> **Answer:** Common commands: mvn clean, compile, test, package, install, verify, deploy, mvn clean install -DskipTests, mvn test -Dtest=LoginTest and mvn dependency:tree.
>
> **Example:** mvn clean test before every regression run.

```
mvn clean
mvn clean test
mvn clean install -DskipTests
mvn test -Dtest=LoginTest
mvn dependency:tree
```

Section 2: High Priority (18 questions)

Frequent practical questions. Answer with the command or pom.xml snippet you would use.

Git: Merge, Rebase, Conflicts & Pull Requests (2)

17. How do you merge branches?

> **Answer:** Switch to the branch that should receive the changes and run git merge with the source branch; resolve conflicts if any and push.
>
> **Example:** Merge feature/search-tests into develop.

```
git checkout develop
git pull origin develop
git merge feature/search-tests
git push origin develop
```

18. How do you stash changes in Git and when do you use git stash? **[asked x2 in your bank]**

> **Answer:** git stash saves uncommitted changes temporarily so you can switch branches or pull; bring them back with stash pop or apply.
>
> **Example:** Stash half-written locators to fix an urgent failing test on another branch.

```
git stash push -m "wip checkout locators"
git stash list
git stash pop
```

Git: Fundamentals, Branching & Workflow (4)

19. What is the difference between Git and GitHub?

> **Answer:** Git is the version control tool that runs on your machine. GitHub is a hosting platform for Git repositories with pull requests, issues, Actions and access control.
>
> **Example:** Git tracks the changes; GitHub hosts the team's shared test repo.

20. What is the difference between a local and a remote repository?

> **Answer:** A local repository is on your machine (the .git folder). A remote repository is hosted on a server (usually named origin). You push and pull to sync them.
>
> **Example:** Commit locally during the day and push to origin when the work is ready.

```
git remote -v
git push origin main
```

21. What is .gitignore and why does QA use it? **[asked x2 in your bank]**

> **Answer:** .gitignore lists files Git must not track: build output (target/), reports, logs, screenshots, IDE files, downloaded drivers and files with secrets. It keeps the repo clean and safe.
>
> **Example:** target/ and credentials.properties are ignored.

```
target/
test-output/
*.class
.idea/
*.log
screenshots/
config/secrets.properties
```

22. How do you trigger builds using Git?

> **Answer:** Use a webhook from GitHub/GitLab to Jenkins (or poll the repo) so a push or pull request starts the pipeline; in GitHub Actions use the on: push trigger.
>
> **Example:** A push to develop runs the smoke tests automatically.

```
on:
  push:
    branches: [ develop ]
  pull_request:
```

Git: Everyday Commands, Undo & History (4)

23. What are the basic Git commands (init, clone, status, add, commit, push, pull)? How do you clone a repository from GitHub? **[asked x4 in your bank]**

> **Answer:** Basic flow: git clone (or init), status, add, commit, push, pull.
>
> **Example:** Clone the framework repo, add a test, commit and push.

```
git clone https://github.com/team/shop-automation.git
git status
git add .
git commit -m "Add cart tests"
git push origin main
git pull origin main
```

24. How do you create and switch to a new branch, and check which branch you are on? **[asked x3 in your bank]**

> **Answer:** Create and switch with git checkout -b (or git switch -c), list branches with git branch (current one has *) and check with git status.
>
> **Example:** Start work on feature/wishlist-tests.

```
git checkout -b feature/wishlist-tests     # or: git switch -c feature/wishlist-tests
git branch
git status
```

25. How do you undo the last commit but keep the changes?

> **Answer:** Use git reset --soft HEAD~1 to undo the last commit and keep the changes staged (or --mixed to keep them unstaged). Use it only for commits that are not pushed.
>
> **Example:** Undo a commit made with the wrong message and recommit.

```
git reset --soft HEAD~1
```

26. How do you view commit history?

> **Answer:** Use git log; common options: --oneline, --graph, -n 5, --author, -p (changes) and git show for one commit.
>
> **Example:** Check who changed the checkout locators last week.

```
git log --oneline -n 10
git log --graph --oneline --all
git log --author="Asha" -p
git show <commit-id>
```

Maven in Automation Projects (TestNG, Profiles, Jenkins) (2)

27. What are profiles in Maven and how do you use them for different environments?

> **Answer:** A Maven profile is a set of configuration (properties, dependencies, plugins) activated with -P. Use one profile per environment and pass the value to the tests through Surefire system properties.
>
> **Example:** mvn test -Pqa uses QA settings; -Puat uses UAT.

```
<profiles>
  <profile><id>qa</id><properties><env>qa</env></properties></profile>
  <profile><id>uat</id><properties><env>uat</env></properties></profile>
</profiles>
<!-- Surefire: <systemPropertyVariables><env>${env}</env></systemPropertyVariables> -->
<!-- test code: System.getProperty("env") ; run: mvn clean test -Pqa -->
```

28. How do you use Maven with Jenkins?

> **Answer:** Configure JDK and Maven in Jenkins tools, pull the repo and run mvn clean test in a Maven job or pipeline step, then publish the test reports.
>
> **Example:** Nightly Jenkins pipeline runs the regression with Maven.

```
pipeline {
  agent any
  tools { maven 'Maven3'; jdk 'JDK17' }
  stages { stage('Test') { steps { sh 'mvn -B clean test -Denv=qa' } } }
}
```

Maven: Fundamentals, pom.xml & Dependencies (2)

29. What are Maven repositories and their types (local, central, remote)? **[asked x2 in your bank]**

> **Answer:** Local repository: ~/.m2/repository on your machine. Central: the default public repository (Maven Central). Remote: a company or third-party repository such as Nexus or Artifactory. Maven checks local first, then remote.
>
> **Example:** The framework's internal utilities jar is stored in the company Nexus.

```
<repositories>
  <repository>
    <id>company-nexus</id>
    <url>https://nexus.shop.com/repository/maven-public/</url>
  </repository>
</repositories>
```

30. What is a parent POM and how does inheritance work in Maven? **[asked x2 in your bank]**

> **Answer:** A parent POM holds shared settings (versions, plugins, properties, dependencyManagement) that child projects inherit with the <parent> element, so versions are defined once.
>
> **Example:** UI-tests and API-tests modules inherit the same Selenium and TestNG versions.

```
<parent>
  <groupId>com.shop</groupId>
  <artifactId>automation-parent</artifactId>
  <version>1.0</version>
</parent>
```

Maven: Lifecycle, Commands & Plugins (4)

31. How do you run tests using Maven?

> **Answer:** Run mvn test; limit it with -Dtest for a class or method, -Dgroups for TestNG groups or -DsuiteXmlFile for a suite.
>
> **Example:** Run only the login test after a locator fix.

```
mvn test
mvn test -Dtest=LoginTest
mvn test -Dtest=LoginTest#validLogin
mvn test -Dgroups=smoke
```

32. What is the difference between build and clean, and between install and deploy? **[asked x2 in your bank]**

> **Answer:** clean deletes the target folder (old build output); build phases such as package compile, test and package the project. install copies the artifact to the local repository (~/.m2); deploy copies it to a remote repository such as Nexus so others can use it.
>
> **Example:** mvn clean install makes the framework jar available to other local projects; deploy shares it with the team.

33. What is the purpose of Maven plugins? What is the Surefire plugin? **[asked x2 in your bank]**

> **Answer:** Maven does its work through plugins (compiler, Surefire, jar and so on), whose goals are attached to lifecycle phases. Surefire runs the tests in the test phase and supports suite files, groups, parallel runs and reruns.
>
> **Example:** Surefire runs testng.xml and reruns failed tests once.

```
<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <configuration>
    <suiteXmlFiles><suiteXmlFile>testng.xml</suiteXmlFile></suiteXmlFiles>
    <rerunFailingTestsCount>1</rerunFailingTestsCount>
  </configuration>
</plugin>
```

34. How do you exclude tests in a Maven build?

> **Answer:** Use Surefire excludes or excludedGroups, a negated -Dtest pattern, -DskipTests (compiles but skips running) or -Dmaven.test.skip=true (skips compiling tests too).
>
> **Example:** Skip the slow regression group in the quick PR build.

```
mvn test -Dtest='!SlowTest'
mvn package -DskipTests
mvn test -DexcludedGroups=regression
```

Section 3: Medium Priority (7 questions)

Occasionally asked, usually as follow-ups to a Git or Maven answer.

Git: Fundamentals, Branching & Workflow (3)

35. What are Git tags and how does QA use them?

> **Answer:** A tag labels a specific commit, usually a release or version. QA uses tags to mark the code version that was tested and to reproduce a regression suite later.
>
> **Example:** Tag v2.3 for the release that passed the full regression.

```
git tag -a v2.3 -m "Regression passed for release 2.3"
git push origin v2.3
git checkout v2.3
```

36. What is the difference between GitHub and GitLab?

> **Answer:** Both host Git repositories. GitHub (Microsoft) has a huge community and GitHub Actions; GitLab includes built-in CI/CD, issue boards and strong self-hosting options.
>
> **Example:** GitLab CI runs the pipeline directly from the repo without a separate tool.

37. How do you fork a repository and set an upstream remote? **[asked x2 in your bank]**

> **Answer:** Fork creates your own copy of a repository on the server. Clone your fork and add the original repository as upstream to keep it updated, then open a PR from your fork.
>
> **Example:** Fork a shared test library, fix a bug and send a PR.

```
git clone https://github.com/me/shop-automation.git
git remote add upstream https://github.com/team/shop-automation.git
git fetch upstream
git merge upstream/main
```

Git: Everyday Commands, Undo & History (3)

38. How do you delete or rename a branch? **[asked x2 in your bank]**

> **Answer:** Delete with git branch -d (or -D to force) and remove a remote branch with git push origin --delete. Rename with git branch -m.
>
> **Example:** Clean up merged feature branches.

```
git branch -d feature/cart-tests
git push origin --delete feature/cart-tests
git branch -m old-name new-name
```

39. How do you see the differences between two commits?

> **Answer:** Use git diff with the two commit ids (or branches); --stat gives a summary and git show shows one commit.
>
> **Example:** See what changed between the last two releases.

```
git diff <commit1> <commit2>
git diff --stat main..feature/search-tests
```

40. How do you see who made changes to a file?

> **Answer:** Use git blame <file> to see who last changed each line, and git log -p <file> for the full change history.
>
> **Example:** Find who changed the payment locator that broke the test.

```
git blame src/test/java/pages/PaymentPage.java
git log -p --follow src/test/java/pages/PaymentPage.java
```

Maven in Automation Projects (TestNG, Profiles, Jenkins) (1)

41. How do you integrate Cucumber in a Maven project?

> **Answer:** Add the cucumber-java and cucumber-testng (or JUnit platform) dependencies, put feature files under src/test/resources and step definitions and a runner under src/test/java, then run mvn test (use -Dcucumber.filter.tags for tags).
>
> **Example:** Run only @smoke scenarios from Maven.

```
<dependency><groupId>io.cucumber</groupId><artifactId>cucumber-java</artifactId>
  <version>7.18.0</version></dependency>
<dependency><groupId>io.cucumber</groupId><artifactId>cucumber-testng</artifactId>
  <version>7.18.0</version></dependency>
<!-- mvn test -Dcucumber.filter.tags="@smoke" -->
```